

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

Лекция 4

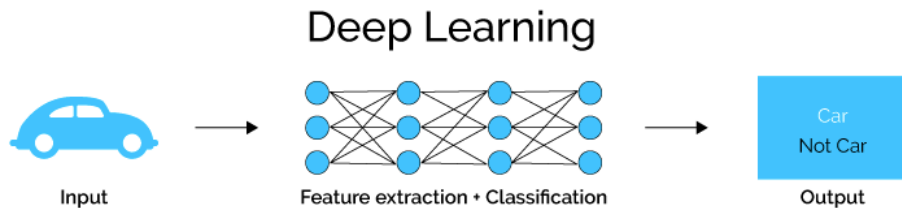
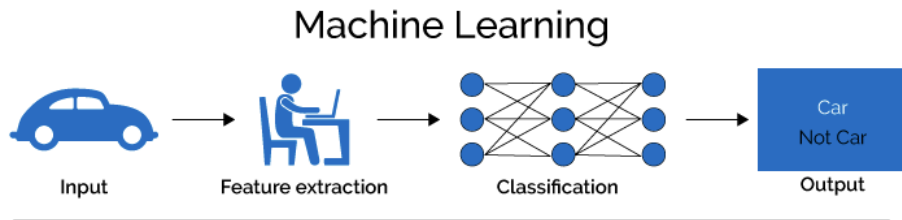
22 сентября 2025

Классический ML

Линейная регрессия — предсказание числового значения

Логистическая регрессия — бинарная классификация

Деревья решений — деление пространства признаков по правилам
«если—то»



Ограничения классических методов

Работают хорошо на **табличных** данных (Excel-таблицы, числовые признаки)

Трудности:

- Обработка изображений (пиксели — миллионы признаков)
- Речь и аудио (сигналы, временные ряды)
- Тексты (слова, контекст, смысл)

Нужен другой подход для сложных, высокоразмерных данных

Особенности работы с изображениями и текстом

1. Избыточное количество информации

Пиксели фона на фотографии и предлоги/союзы в тексте несут малое количество значимой информации

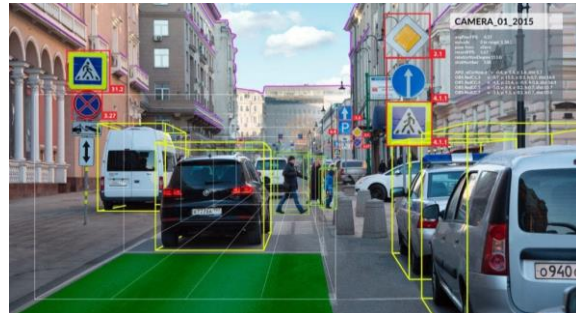
2. Соседние признаки связаны друг с другом

Соседние пиксели на изображении принадлежат одному объекту, а рядом стоящие слова в предложении связаны по смыслу

Справиться с большим числом признаков и учесть их порядок могут нейронные сети

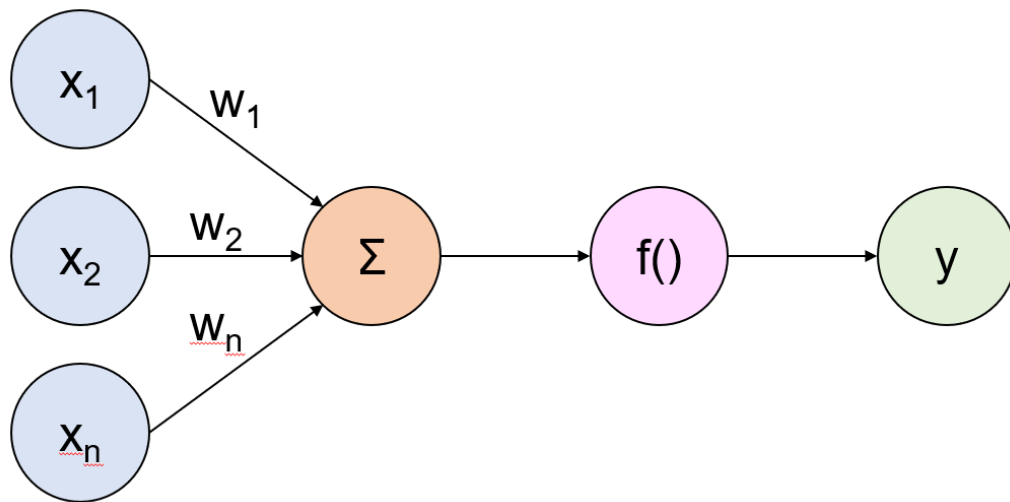
Современные примеры

- Распознавание лиц (Face ID, камеры)
- GPT и чат-боты (работа с текстом)
- Автономные автомобили (распознавание объектов на дороге)
- Медицинская диагностика (анализ снимков)
- Генерация изображений (Stable Diffusion, DALL-E)



Что такое нейрон?

Нейрон — это вычислительная единица, которая получает информацию, производит над ней простые вычисления и передает ее дальше



Структура нейрона

x_1, x_2, x_n – входы в нейрон

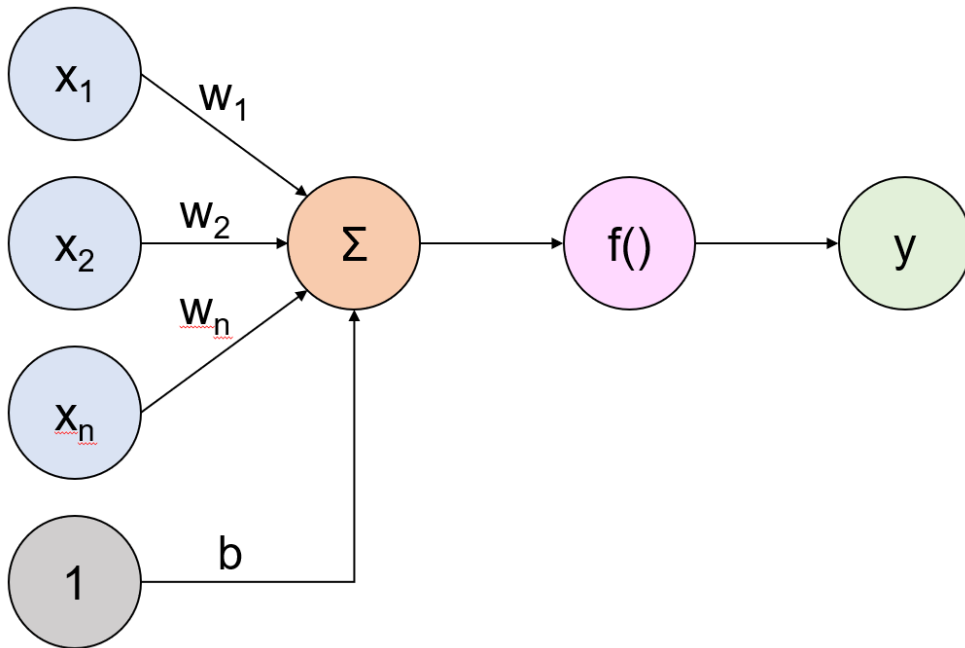
w_1, w_2, w_n – веса нейрона

Σ – сумматор

$f(x)$ – функция активации

y – выход нейрона

b – смещение



Структура нейрона

x_1, x_2, x_n – входы в нейрон

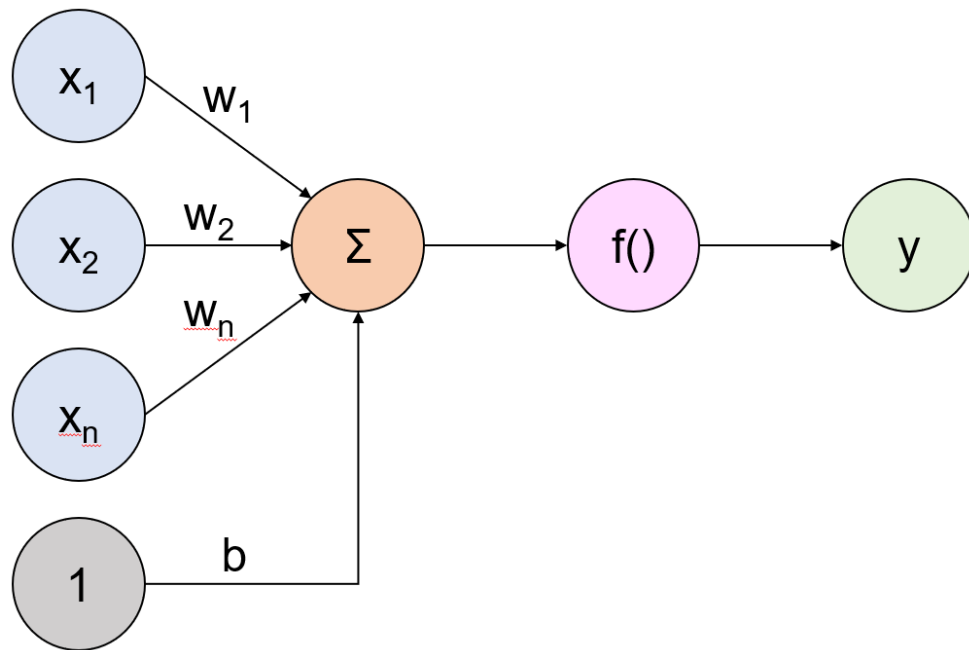
w_1, w_2, w_n – веса нейрона

Σ – сумматор

$f(x)$ – функция активации

y – выход нейрона

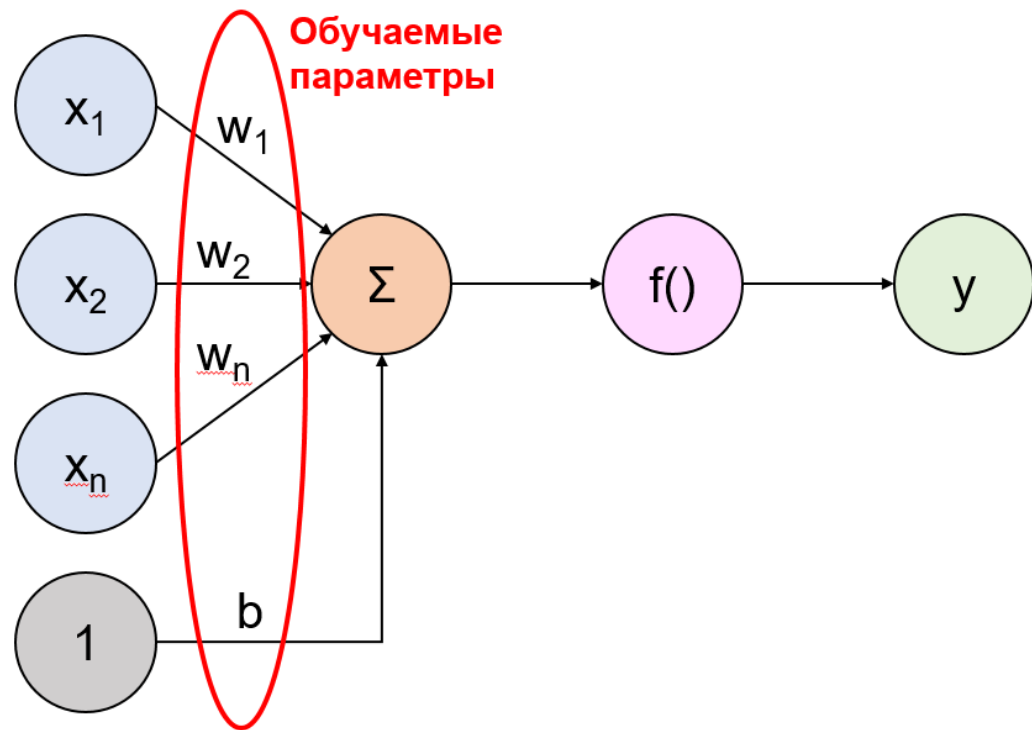
b – смещение



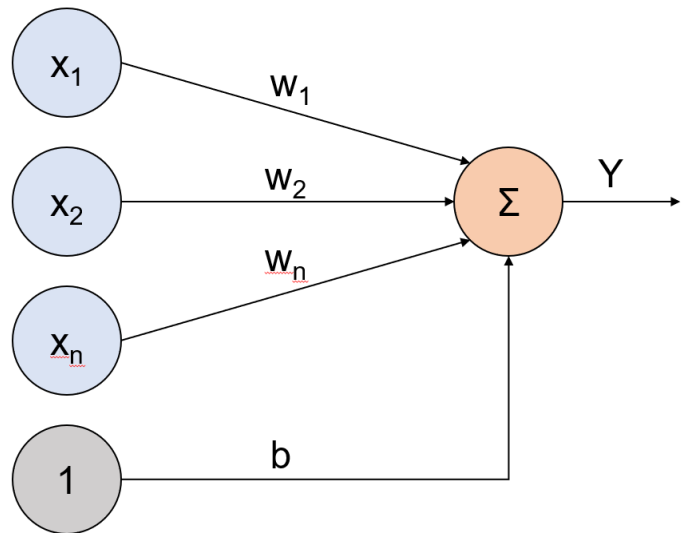
$$f(x_1w_1 + x_2w_2 + \dots + x_nw_n + b) = f\left(\sum_{i=1}^n x_iw_i + b\right)$$

Структура нейрона

$w_1, w_2 \dots w_n, b$ —
настраиваемые
параметры нейрона

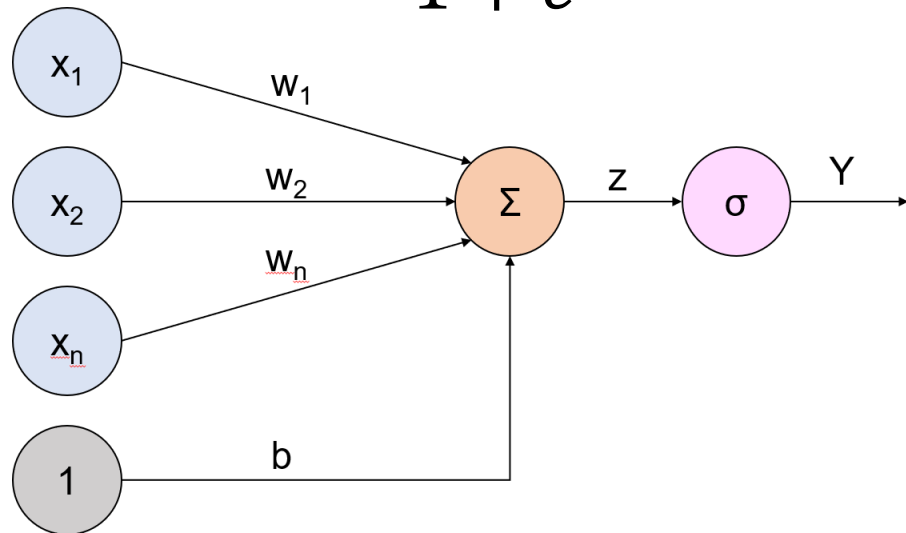


Сигмоида



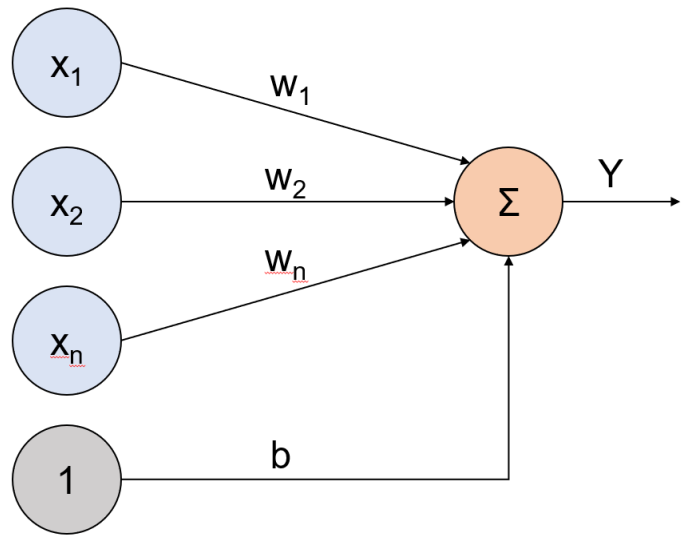
$$Y = \sum_{i=1}^n x_i w_i + b$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

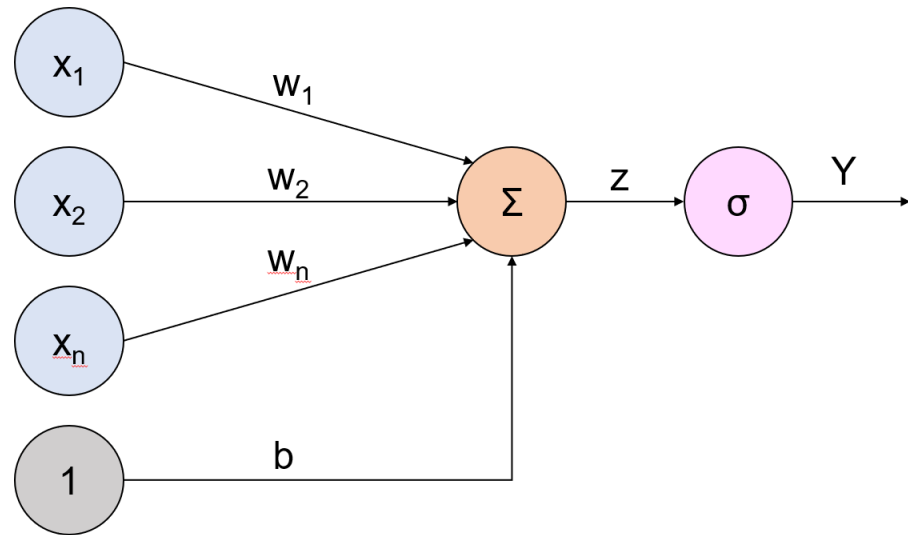


$$Y = \sigma(\sum_{i=1}^n x_i w_i + b)$$

Сигмоида



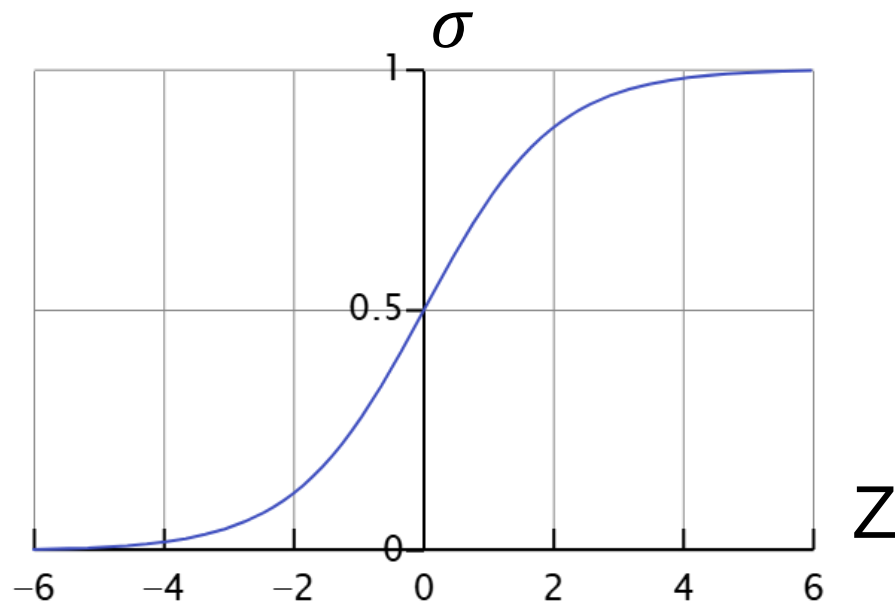
Линейная
регрессия
 $Y \in [-\infty; +\infty]$



Логистическая
регрессия
 $Y \in [0; 1]$

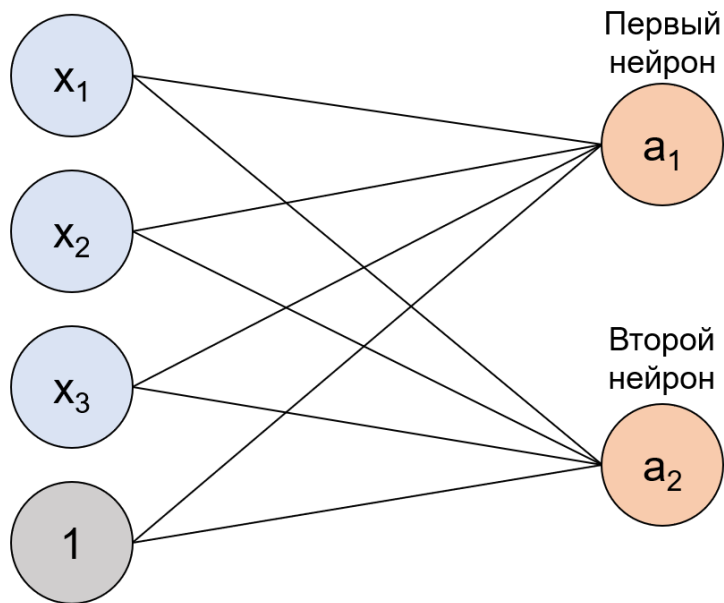
Сигмоида

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



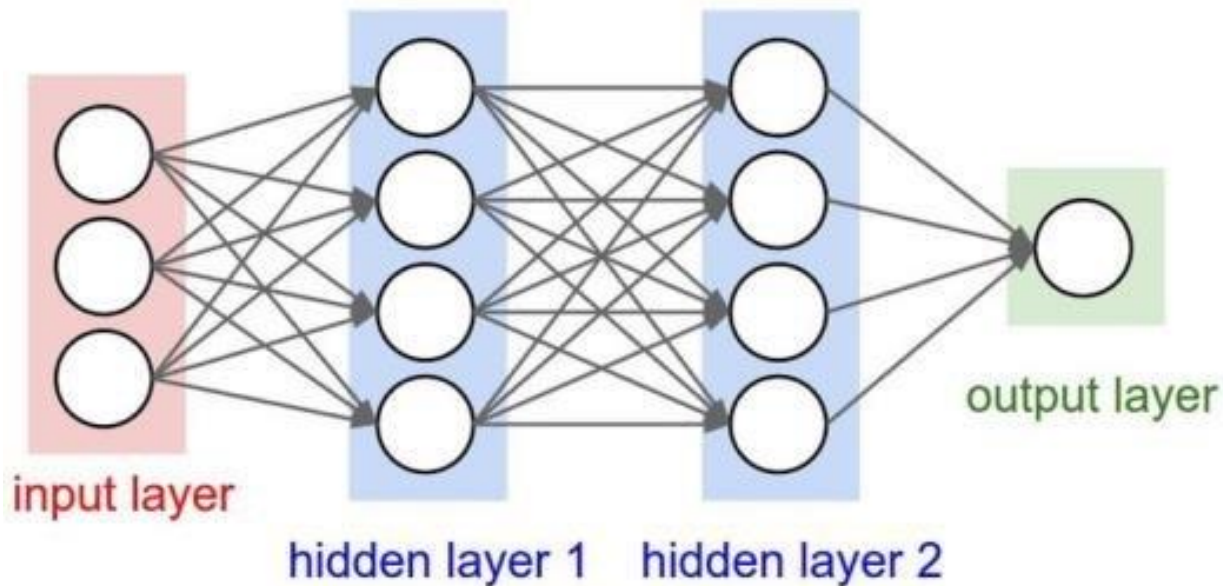
Полносвязная сеть

Слои, в которых все входы соединены со всеми нейронами, называются **полносвязными слоями** (англ. fully connected layers)



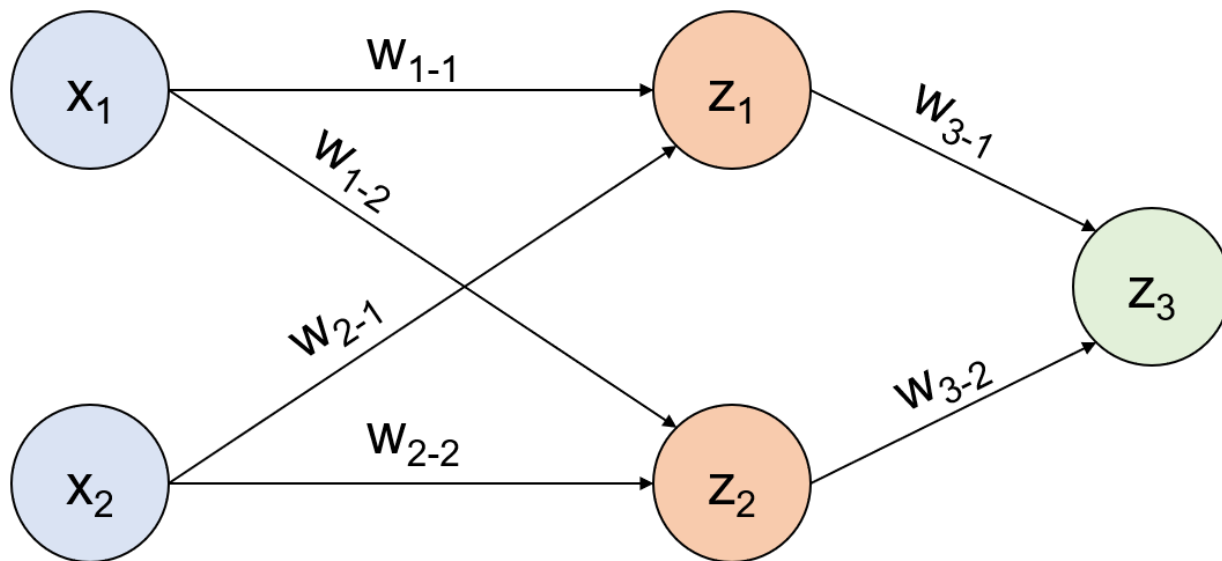
Скрытые слои

Скрытыми слоями (англ. hidden layers) называются все слои, кроме входного (input) и выходного (output)

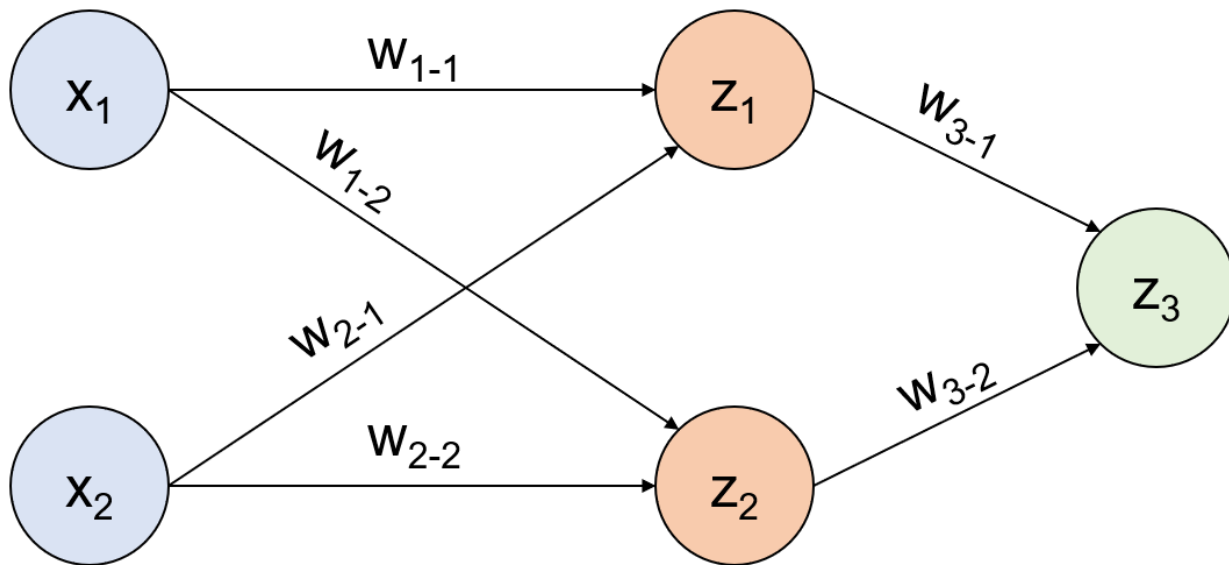


Функция активации

В полносвязных сетях после каждого нейрона, кроме последнего, есть **функция активации**. Для чего она нужна?



Функция активации



$$Z_1 = X_1 * W_{1-1} + X_2 * W_{1-2}$$

$$Z_2 = X_1 * W_{2-1} + X_2 * W_{2-2}$$

$$Z_3 = Z_1 * W_{3-1} + Z_2 * W_{3-2}$$

$$Z_3 = (X_1 * W_{1-1} + X_2 * W_{1-2}) * W_{3-1} + (W_{2-1} * X_1 + W_{2-2} * X_2) * W_{3-2}$$

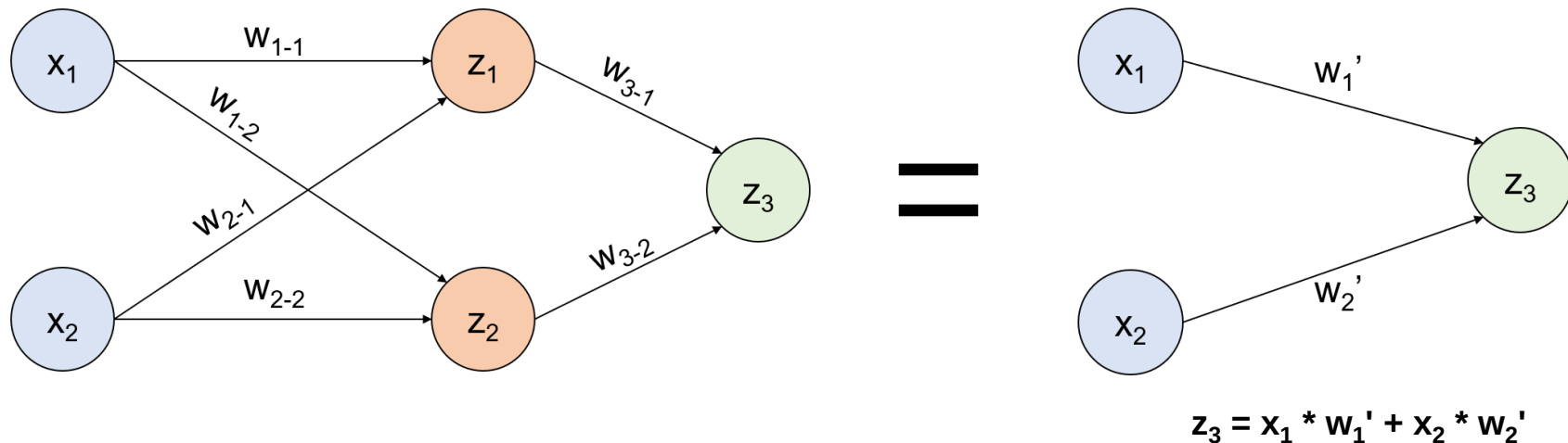
$$Z_3 = X_1 * (W_{1-1} * W_{3-1} + W_{2-1} * W_{3-2}) + X_2 * (W_{1-2} * W_{3-1} + W_{2-2} * W_{3-2})$$

$$W_1' = W_{1-1} * W_{3-1} + W_{2-1} * W_{3-2}$$

$$W_2' = W_{1-2} * W_{3-1} + W_{2-2} * W_{3-2}$$

$$Z_3 = X_1 * W_1' + X_2 * W_2'$$

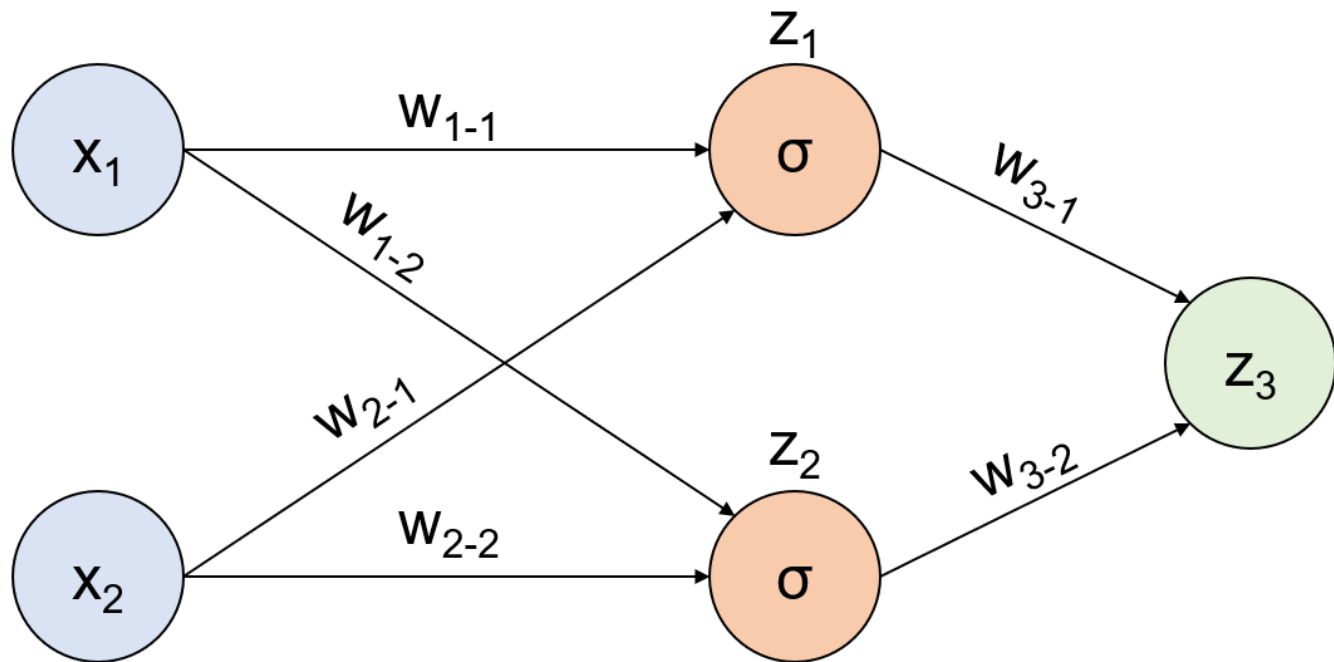
Функция активации



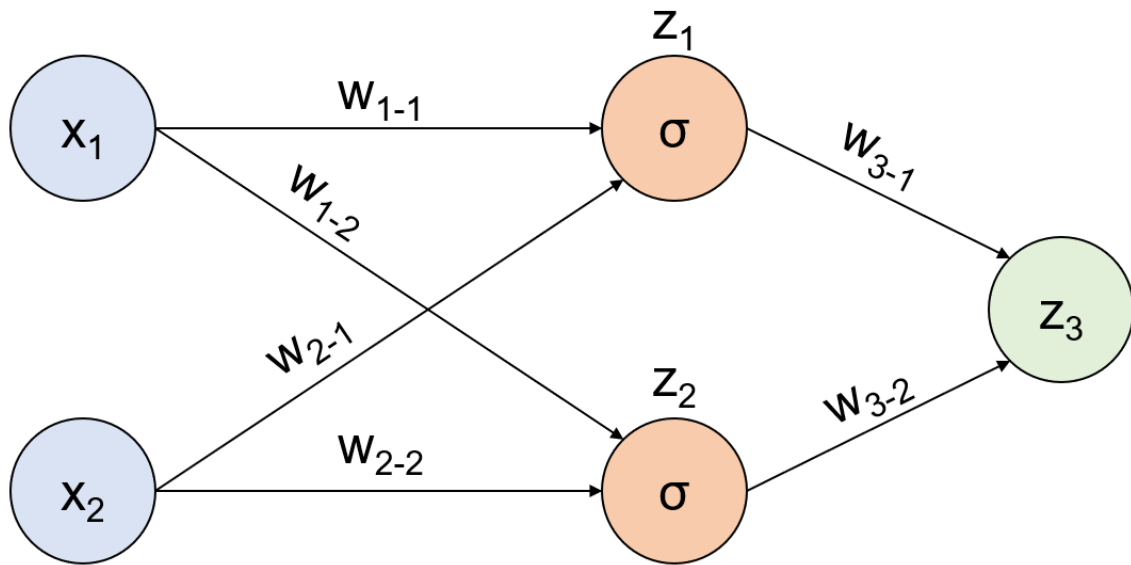
Многослойная нейронная сеть без функции активации — это сеть с одним нейроном

Функция активации

Введем сигмоидную функцию активации



Функция активации



$$z_1 = \sigma(x_1 * w_{1-1} + x_2 * w_{1-2})$$

$$z_2 = \sigma(x_1 * w_{2-1} + x_2 * w_{2-2})$$

$$z_3 = z_1 * w_{3-1} + z_2 * w_{3-2}$$

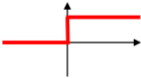
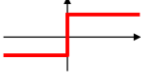
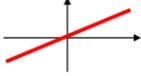
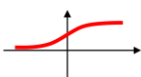
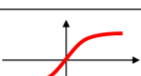
$$z_3 = \sigma(x_1 * w_{11} + x_2 * w_{12}) * w_{31} + \sigma(w_{21} * x_1 + w_{22} * x_2) * w_{32}$$

Из-за сигмоид вынести за скобки x_1 и x_2 нельзя, а значит это уже не просто один нейрон. **Сигмоида позволяет сделать сеть сложнее**

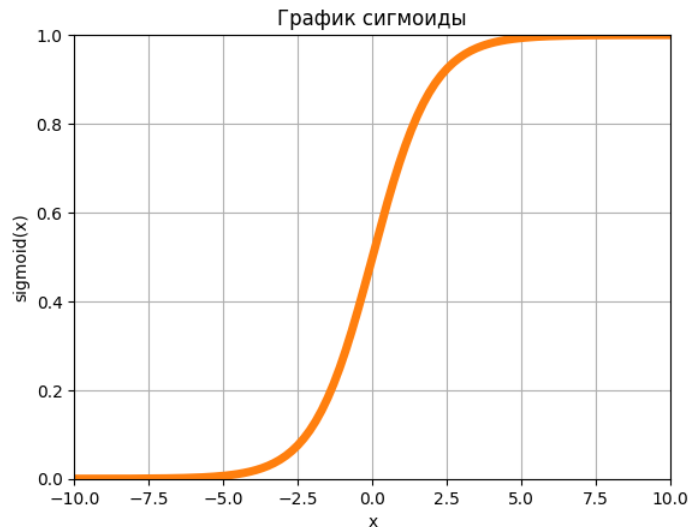
Для чего нужны функции активации?

- Линейные модели описывают только прямые зависимости между признаками и целевой переменной
- Если убрать активацию, многослойная сеть превращается в одну линейную модель
- Нелинейные функции активации позволяют сети строить изогнутые границы решений
- Это делает возможным решение задач, где классы невозможно разделить прямой линией

Различные функции активации

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

Различные функции активации. Сигмоида

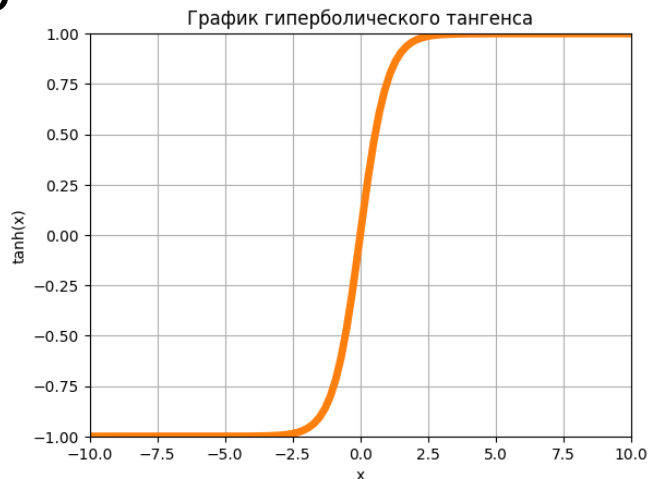


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Сигмоидная функция хорошо подходит в качестве активации на выходном слое нейросети для задач бинарной классификации, так как вместе с правильной функцией потерь (log-loss) она позволяет интерпретировать выход как вероятность принадлежности к классу.

Однако у неё есть недостаток, из-за которого её редко используют в скрытых слоях: при больших по модулю значениях аргумента $|x|$ функция $\sigma(x)$ изменяется очень слабо, а её производная $\sigma'(x)$ становится близкой к нулю. Это приводит к тому, что градиент функции потерь $\nabla Q(w)$ почти обнуляется, и обучение, основанное на градиентном спуске, может «замереть»

Различные функции активации. Гиперболический тангенс

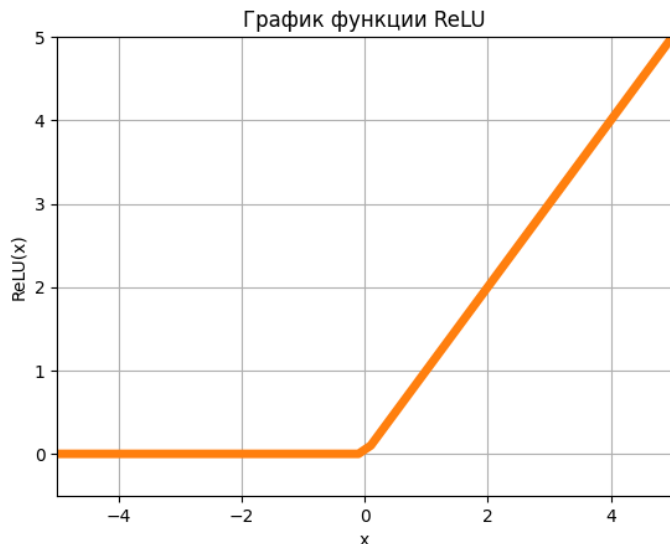


$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Гиперболический тангенс по своей сути похож на сигмоидальную функцию и также применяется в нейронных сетях. Он преобразует входные данные в значения от -1 до 1 , что делает его удобным, например, для задач регрессии: результаты, близкие к -1 , трактуются как отрицательные, близкие к 1 — как положительные, а около нуля — как нейтральные.

В отличие от сигмиды, тангенс обладает более плавным переходом, что помогает сети эффективнее выявлять сложные зависимости в данных. Кроме того, его производная является непрерывной и гладкой, что делает возможным применение оптимизационных методов, основанных на градиенте

Различные функции активации. ReLU



$$\text{ReLU}(x) = \max(0, x)$$

ReLU (Rectified Linear Unit) — это простая и популярная функция активации в глубоком обучении, которая преобразует входные значения в диапазон от 0 до $+\infty$: если вход меньше или равен нулю, результат равен нулю, а если больше нуля — возвращается само значение. Графически это линейная функция с обнулением в точке 0. ReLU вычисляется быстрее сигмоиды, так как не требует сложных операций, и помогает справиться с проблемой затухающего градиента при обучении.

В то же время у неё есть ограничения: некоторые нейроны могут "умирать", оставаясь навсегда неактивными при отрицательных входах, а также функция несимметрична относительно нуля, что может вызывать смещение активаций в положительную область. Для уменьшения этих эффектов применяют модификации вроде Leaky ReLU или ELU

Различные функции активации. Leaky ReLU



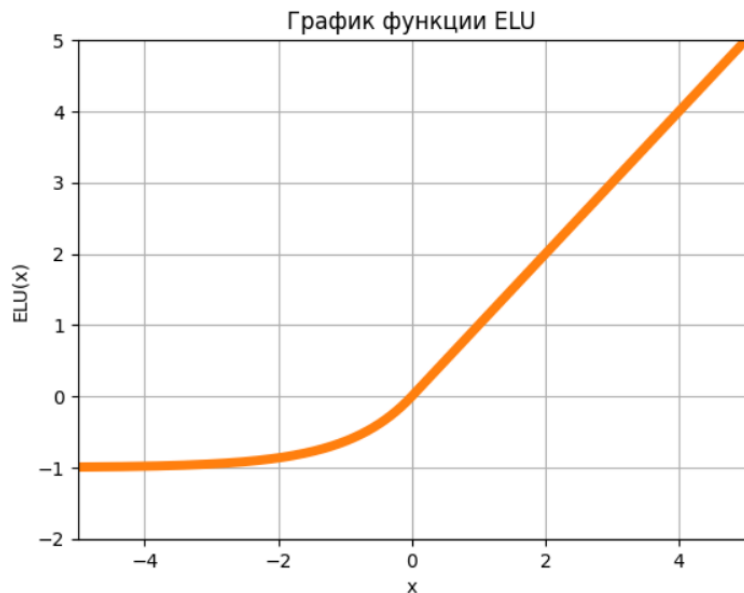
$$\text{LeakyReLU}(x) = \max(\alpha x, x)$$

α — отрицательный уклон.

Небольшое положительное число (обычно 0.01)

Leaky ReLU — это модификация функции ReLU, используемая в нейронных сетях для введения нелинейности. В отличие от обычной ReLU, которая обнуляет все отрицательные входы и может приводить к «умиранию» нейронов, Leaky ReLU пропускает небольшое отрицательное значение (умноженное на коэффициент α). Это позволяет сохранять градиент и улучшает сходимость модели, делая обучение быстрее и стабильнее

Различные функции активации. ELU

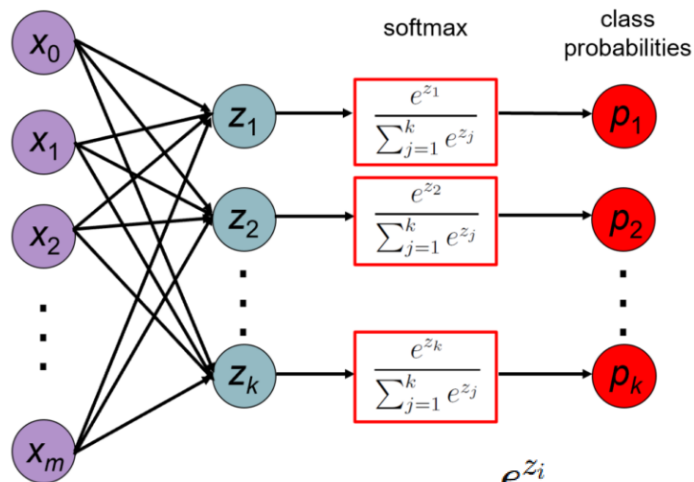


$$f(x) = \begin{cases} x, & x > 0, \\ \alpha (e^x - 1), & x \leq 0, \end{cases}$$

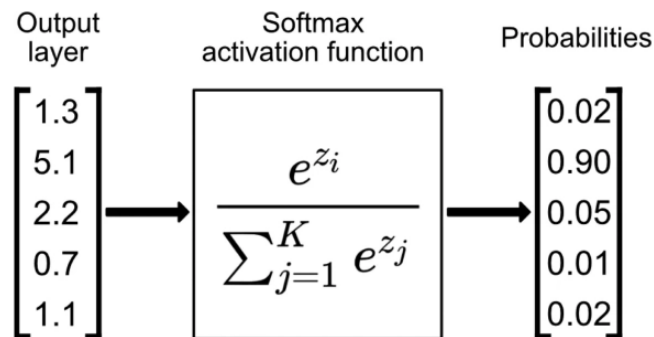
$\alpha > 0$ — параметр, задающий «глубину» отрицательной части (обычно $\alpha = 1$)

ELU (Exponential Linear Unit) работает как ReLU для положительных входов ($f(x)=x$), но для отрицательных использует экспоненту $f(x)=\alpha(e^x-1)$, что снижает риск «мертвых нейронов» и ускоряет обучение. Благодаря гладкости ELU помогает бороться с проблемой взрывающегося градиента и делает обучение более стабильным, однако её эффективность зависит от конкретной задачи, поэтому выбор активации всегда требует экспериментов

Различные функции активации. Softmax



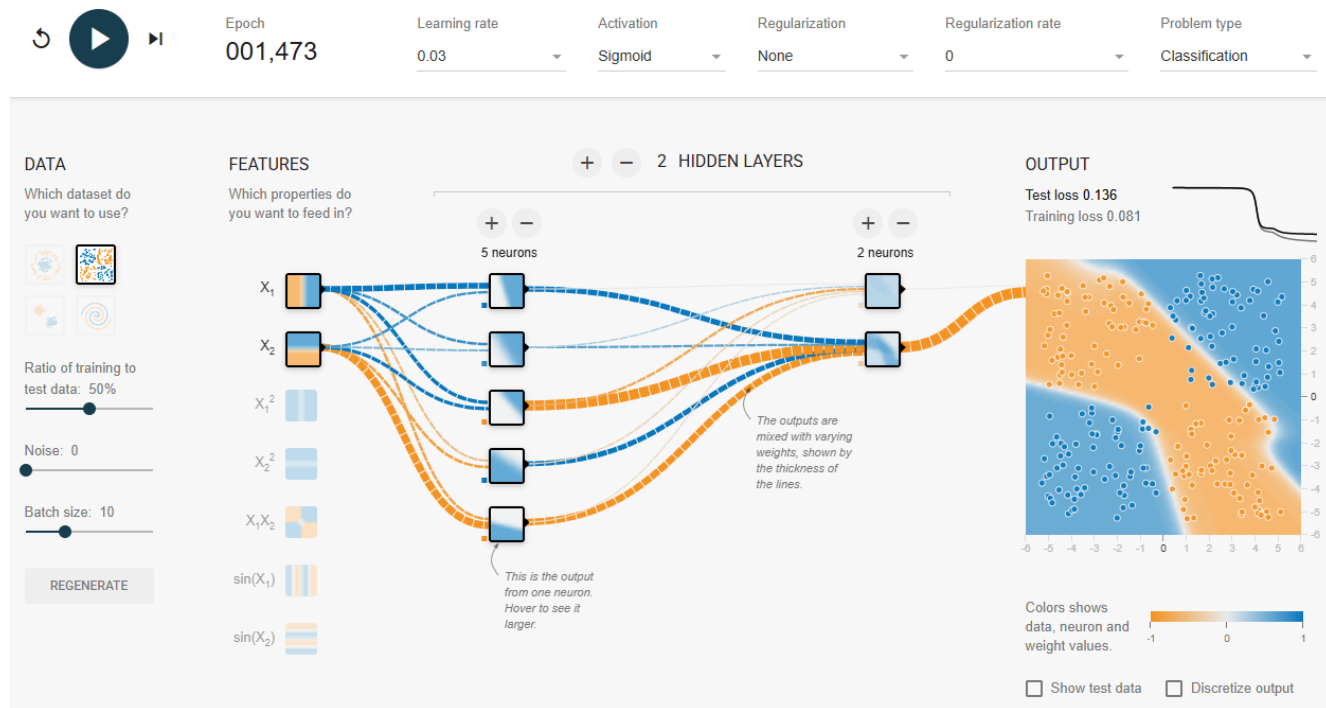
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, 2, \dots, K \quad K — \text{количество классов}$$



Softmax — это функция активации, которая преобразует набор чисел в вероятностное распределение по классам. Для каждого значения она вычисляет экспоненту и нормирует её на сумму экспонент всех элементов, в результате чего выходные значения лежат в диапазоне от 0 до 1 и в сумме дают 1. Благодаря этому Softmax часто используют на выходном слое нейронных сетей для многоклассовой классификации, где каждый результат интерпретируется как вероятность принадлежности объекта к соответствующему классу

TensorFlow Playground

TensorFlow Playground — это интерактивный симулятор нейросети прямо в браузере, созданный командой Google. С его помощью можно «пощупать» работу многослойного перцептрона без кода. <https://playground.tensorflow.org/>

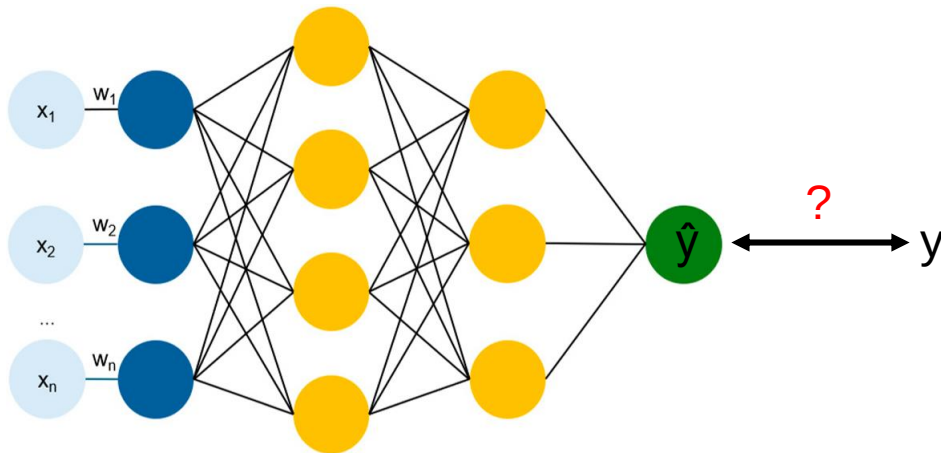


Обучение нейронных сетей

Что значит «обучить нейросеть»?

Настроить параметры (веса W , смещения b) так, чтобы сеть правильно предсказывала выход

- Сравниваем предсказание $\hat{y}$ и правильный ответ y
- Измеряем ошибку (функцию потерь)
- Изменяем параметры модели так, чтобы минимизировать ошибку
- Сеть обучается



Функция потерь

Функция потерь — это метрика несоответствия предсказаний сети ($\hat{y}$) и истинных ответов (y)

MSE (среднеквадратичная ошибка)

Для задач регрессии

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

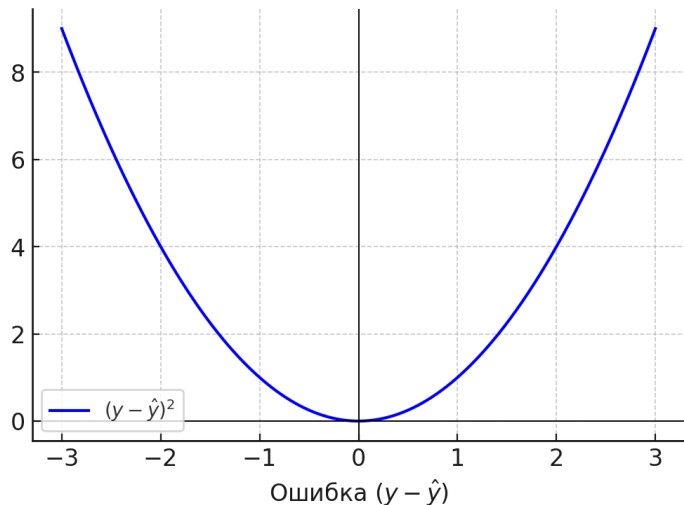
Cross-Entropy (кросс-энтропия)

Для задач классификации

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

MSE (Mean squared error, среднеквадратичная ошибка)

- Используется в задачах регрессии
- Измеряет средний квадрат разницы между предсказанием и правильным ответом
- Усиливает влияние больших ошибок, поэтому чувствителен к выбросам



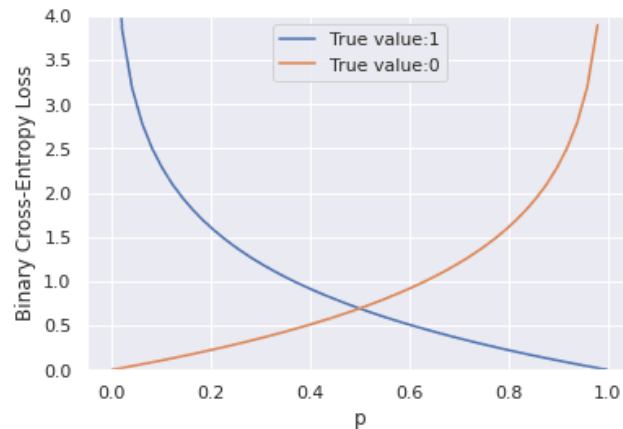
$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Cross-Entropy

- y_i — «правильный ответ» в one-hot (например, $[0,0,1,0,\dots]$)
- $\hat{y}_i$ — вероятность класса, которую выдала сеть (например, из softmax)

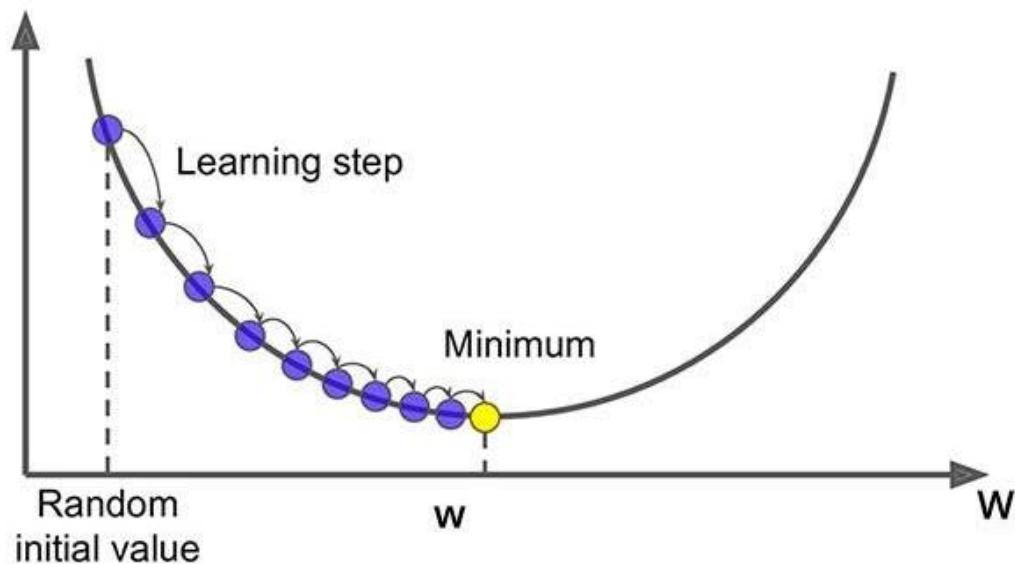
$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

Бинарная кросс-энтропия (BCE) — частный случай кросс-энтропии для бинарной классификации



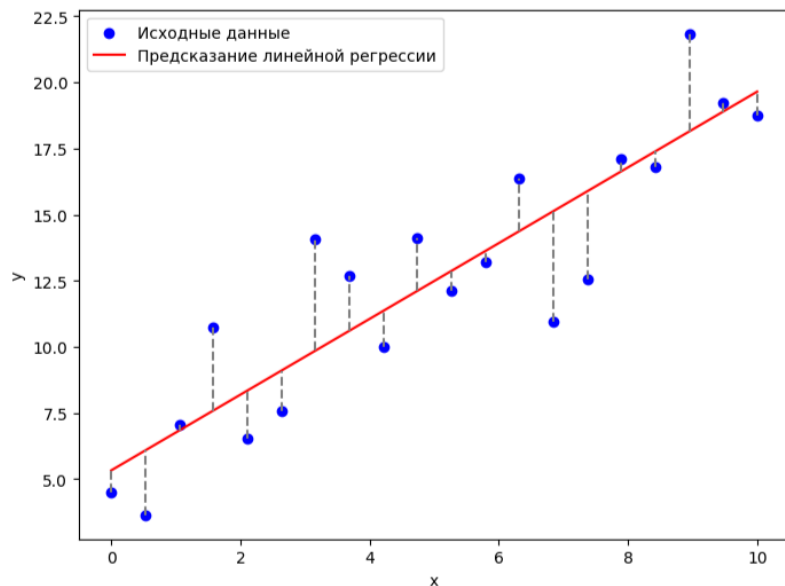
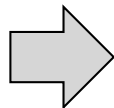
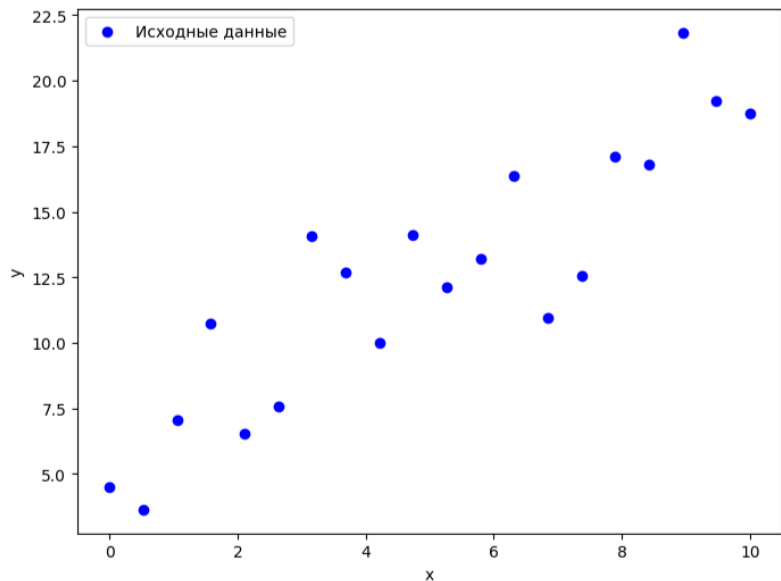
Градиентный спуск

Задача минимизации функции потерь - это сложная задача, которая часто не имеет аналитического (точного) решения. Поэтому для ее решения используют численные методы, а именно, **градиентный спуск**



Градиентный спуск

Рассмотрим градиентный спуск на примере минимизации ошибки в задаче линейной регрессии



Градиентный спуск

Предсказание модели:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_mx_m + b = \sum_{j=1}^m w_jx_j + b$$

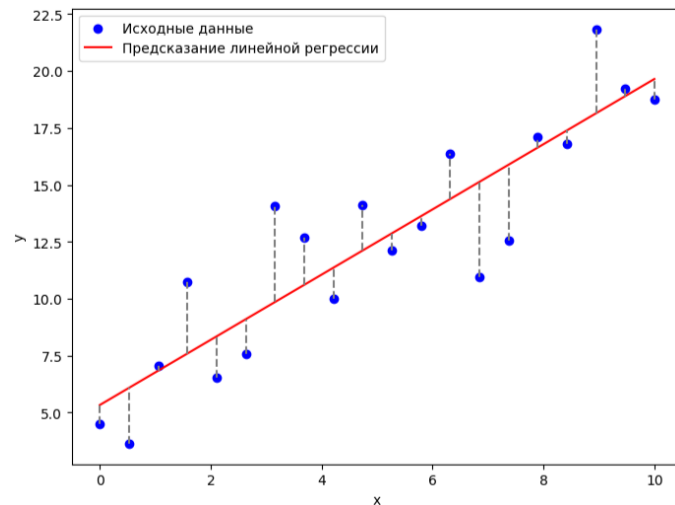
Функция потерь MSE:

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \left(\sum_{j=1}^m w_j x_{ij} + b \right) \right)^2$$

Задача обучения:

$$\min_{w, b} \frac{1}{n} \sum_{i=1}^n \left(y_i - \left(\sum_{j=1}^m w_j x_{ij} + b \right) \right)^2$$



Градиентный спуск

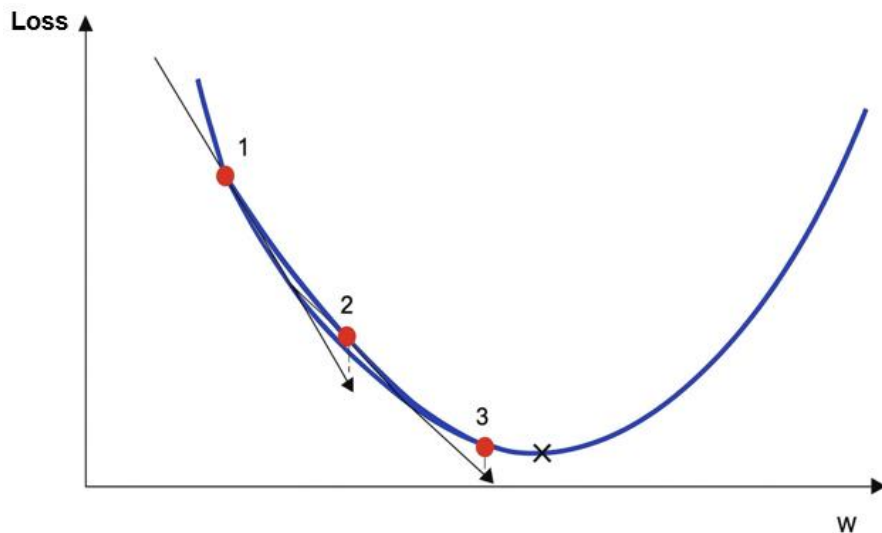
Задачи минимизации ошибки можно решать разными способами:

- Перебор параметров вручную — например, с помощью вложенных циклов на Python, подбирая такие значения параметров, при которых ошибка минимальна
- Аналитически — используя методы математического анализа для нахождения точного решения задачи
- Численными методами — применяя алгоритмы оптимизации, которые позволяют получить решение с высокой (заданной) точностью.

- ☐ Первый способ — перебор параметров — непрактичен: в современных моделях параметры могут исчисляться миллионами, и ручной поиск становится невозможным
- ☐ Второй способ — аналитический — даёт точное решение, но применим не всегда: многие уравнения не поддаются строгому решению
- ✓ Поэтому в машинном обучении используют **численные методы** — универсальные алгоритмы, которые позволяют находить минимумы даже очень сложных функций с высокой точностью.

Градиентный спуск

Градиентный спуск — это метод обучения моделей машинного обучения, который шаг за шагом улучшает их предсказания. Алгоритм вычисляет направление, в котором нужно изменить параметры, чтобы уменьшить ошибку, и корректирует их в эту сторону. Такой процесс повторяется многократно, пока модель не достигнет максимально точных предсказаний.



Градиентный спуск

Дана функция: $f(x, y)$

Тогда градиент функции: $\nabla f = \left\{ \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right\}$

Градиент функции — вектор, состоящий из частных производных функции по всем её аргументам

Найдем градиент функции $f(x, y, z) = 2x - y^2 + yz$

в точке $(x, y, z) = (1, 1, 1)$

Градиентный спуск

$$f(x, y, z) = 2x - y^2 + yz$$

Решение:

$$\frac{\partial f}{\partial x} = 2$$

$$\frac{\partial f}{\partial y} = -2y + z$$

$$\frac{\partial f}{\partial z} = y$$

$$\nabla f = \{2, -2y + z, y\}$$

$$\nabla f(1, 1, 1) = \{2, -1, 1\}$$

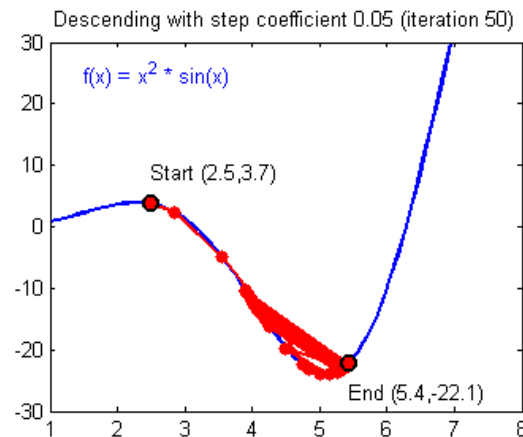
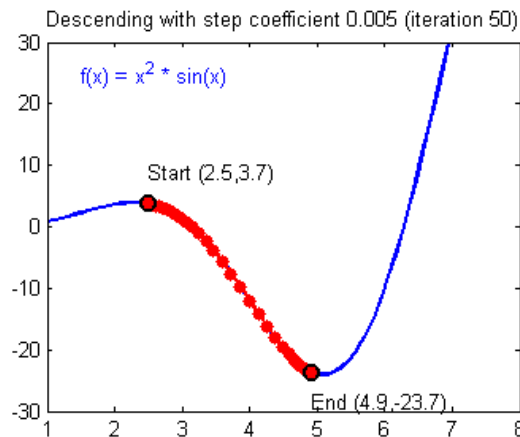
Градиентный шаг (learning rate)

Градиентный шаг η определяет, насколько быстро алгоритм движется к минимуму функции.

Это ключевой гиперпараметр, так как:

- при слишком большом значении η алгоритм может «перескакивать» через минимум и не сходиться вовсе
- при слишком маленьком η обучение будет идти слишком медленно

$$x_{n+1} = x_n - \eta \cdot \nabla f(x_n)$$



Критерии остановки обучения

Градиентный спуск обычно останавливают, когда выполняется одно или несколько условий:

- Веса почти не изменяются между итерациями

$$\|w_{\text{new}} - w_{\text{old}}\| < \varepsilon$$

- Функция потерь перестала заметно уменьшаться

$$|Q(w_{\text{new}}) - Q(w_{\text{old}})| < \varepsilon$$

- Градиент близок к нулю, что указывает на нахождение около минимума

$$\|\nabla Q(w_{\text{new}})\| < \varepsilon$$

- Также алгоритм ограничивают максимальным числом итераций: это не гарантирует нахождение минимума, но позволяет завершить процесс, если обучение длится слишком долго

Недостатки классического градиентного спуска

Градиентный спуск — это численный метод поиска минимумов, применимый даже к очень сложным функциям потерь. Но его реализация требует значительных вычислительных затрат

На каждой итерации приходится:

- вычислять $l \times d$ частных производных (l — число объектов, d — число признаков)
- хранить результаты в памяти

При этом для достижения сходимости обычно нужны сотни или тысячи итераций. Поэтому при больших объемах данных или большом количестве признаков градиентный спуск может работать крайне медленно

Для оптимизации процесса обучения разработаны стохастические вариации градиентного спуска

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD)

Чтобы ускорить обучение, вместо вычисления градиента по всей выборке на каждой итерации можно брать лишь один случайный объект и обновлять параметры только по нему. Такой подход называется стохастическим градиентным спуском (SGD)

Алгоритм:

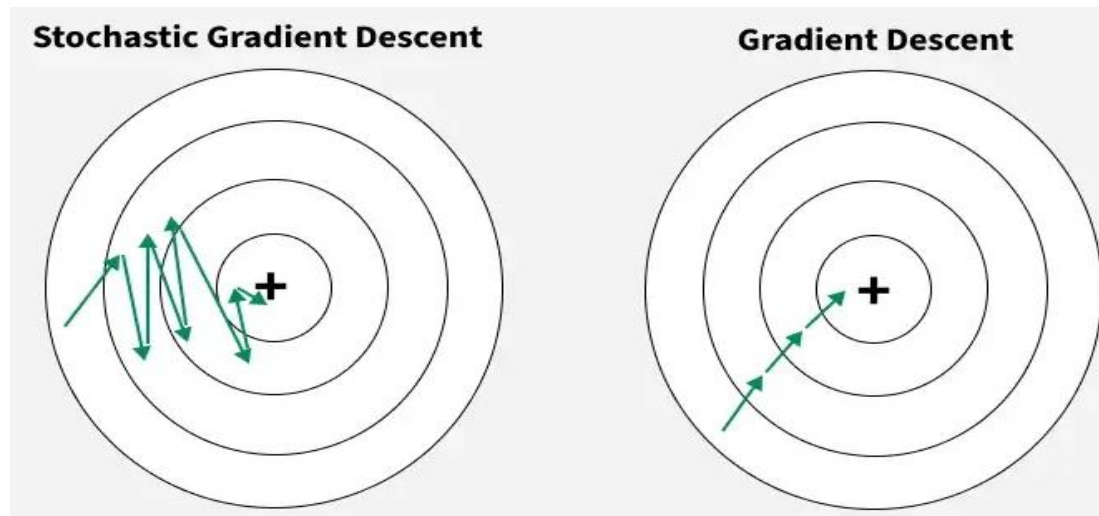
- Случайно задаём начальные параметры w_0
- На каждой итерации:
 1. выбираем случайный объект из выборки
 2. обновляем веса

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla q_i(w_{\text{old}})$$

$\nabla q_i(w_{\text{old}})$ — градиент функции потерь, рассчитанный только на выбранном объекте

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD)

Хотя градиент в SGD на каждой итерации вычисляется приближённо, за счёт случайного выбора и большого числа шагов ошибки усредняются. В итоге метод также сходится к минимуму (как и классический градиентный спуск), хотя для этого обычно требуется больше итераций



Mini-Batch Gradient Descent

Классический градиентный спуск на каждой итерации очень точно определяет направление уменьшения функции, но работает медленно. Стохастический вариант (SGD) — наоборот: быстрый, но вычисляет градиент неточно, так как использует лишь один объект. На практике применяют компромиссный подход — градиентный спуск по мини-батчам (mini-batch GD).

Его идея:

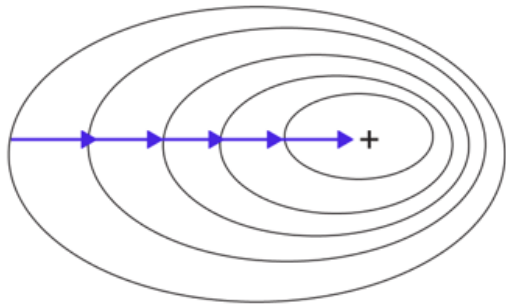
- На каждой итерации выбирается случайная группа объектов (batch) фиксированного размера, например $N = 64$
- Градиент функции потерь вычисляется как среднее по этим объектам:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{1}{N} \sum_{i=1}^N \nabla q_i(w_{\text{old}})$$

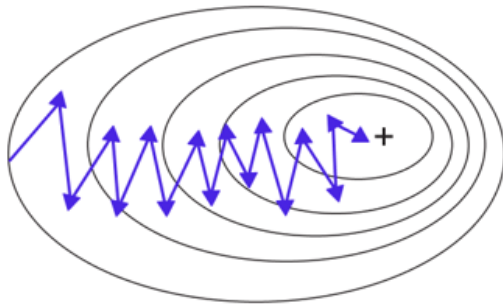
Mini-Batch Gradient Descent

Таким образом, мы получаем баланс: вычислений требуется меньше, чем в полном градиентном спуске, а точность выше, чем у стохастического

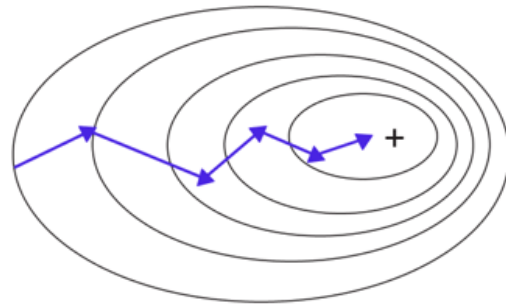
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent

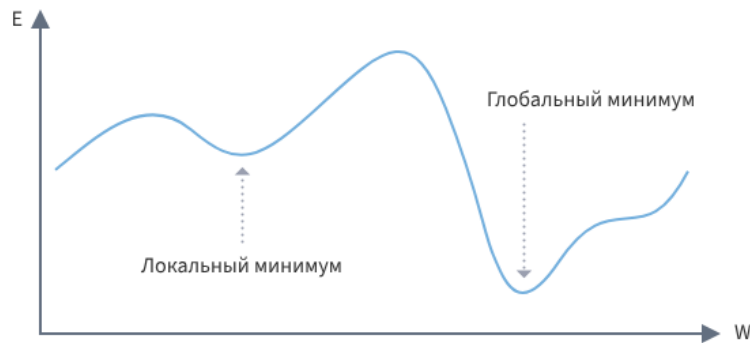


Модификации градиентного спуска

Даже с учетом стохастических оптимизаций у градиентного спуска есть недостатки:

- Он может застревать в локальных минимумах, и нет гарантии нахождения глобального минимума
- Сходимость может быть очень медленной, так как разные веса оптимизируются с разной скоростью: по одним направлениям движение идёт быстро, по другим — медленно, и подобрать подходящий learning rate бывает трудно

На практике применяют **модификации градиентного спуска**, которые помогают преодолеть эти проблемы

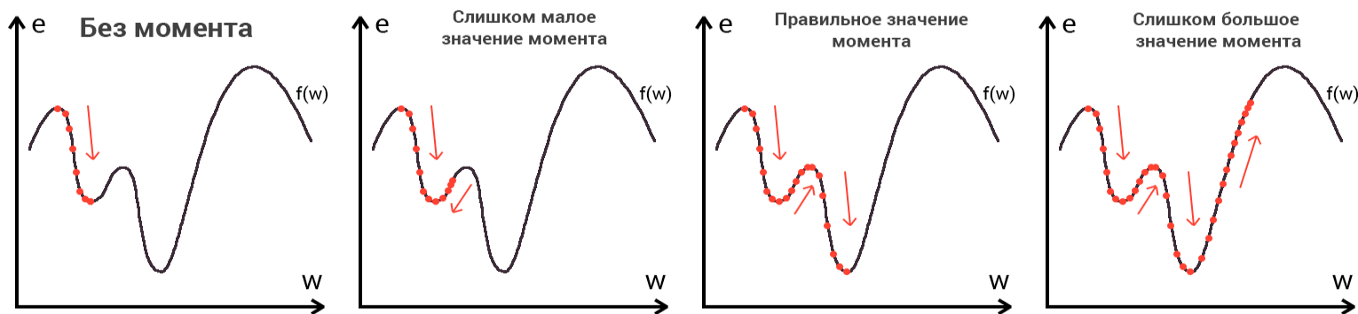


Momentum (градиентный спуск с импульсом)

Идея метода основана на аналогии с мячиком, катящимся по поверхности: он не останавливается мгновенно в первой встреченной впадине, а за счёт накопленной скорости (и массы) может либо немного колебаться возле неё и в итоге остановиться, либо «перепрыгнуть» через локальный минимум и двигаться дальше.

Таким образом, метод способен выходить из мелких локальных минимумов и продолжать поиск глобального.

Ключевой гиперпараметр здесь — **коэффициент момента**, который регулирует накопленную скорость движения. При его правильном выборе метод имеет больше шансов достичь глобального минимума, не застряв в локальных.



Momentum (градиентный спуск с импульсом)

Формула классического градиентного спуска:

$$w_t = w_{t-1} - \eta \nabla Q(w_{t-1})$$

Введем вспомогательный вектор h_t . Он является аналогом скорости в текущий момент времени

$$h_t = \alpha \cdot h_{t-1} + \eta \cdot \nabla Q(w_{t-1})$$

Тогда обновление весов будет выглядеть так:

$$w_t = w_{t-1} - h_t$$

$$w_t = w_{t-1} - (\alpha \cdot h_{t-1} + \eta \cdot \nabla Q(w_{t-1}))$$

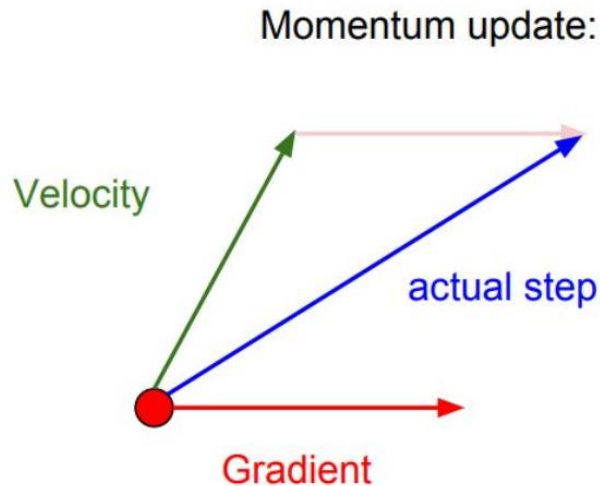
Momentum (градиентный спуск с импульсом)

В обычном градиентном спуске мы «смотрим только на текущий уклон» и шаги могут быть зигзагообразными.

В методе с моментумом мы «накапливаем импульс»: если несколько шагов подряд градиент указывает примерно в одну сторону, движение ускоряется; если направление меняется, инерция помогает «сгладить» колебания.

За счёт этого:

- ускоренное продвижение вдоль направлений с малым градиентом (ускоряет сходимость на «пологих» участках поверхности ошибки)
- снижение вероятности застревания в локальных минимумах благодаря инерции накопленных шагов
- сглаживание колебаний в областях с резкой анизотропией функции потерь, что обеспечивает более стабильное поведение алгоритма



Nesterov Momentum

Процедуру обновления весов в методе моментов можно усовершенствовать — эта модификация называется **Nesterov Momentum**

Проблема классического момента в том, что накопленный импульс иногда уводит траекторию в сторону, отличную от направления градиента, из-за чего теряется время на лишние шаги

Метод Нестерова решает эту проблему: он сначала «заглядывает вперёд», оценивая позицию, куда приведёт текущий импульс, а затем корректирует движение, вычисляя градиент именно в этой точке.

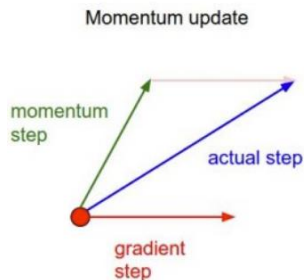
- Обновление весов:

$$w_t = w_{t-1} - h_t$$

- Скорость с поправкой:

$$h_t = \alpha h_{t-1} + \eta \nabla Q(w_{t-1} - \alpha h_{t-1})$$

В методе Нестерова градиент берётся в «заглянутой вперёд» точке $w_{t-1} - \alpha h_{t-1}$, то есть там, куда мы пришли бы, если бы сделали шаг только по импульсу



AdaGrad (Adaptive Gradient)

Итак, проблему застревания в локальных минимумах удалось смягчить с помощью моментов.

Но остаётся другая трудность — разные скорости сходимости по координатам: по одним весам шаги к минимуму происходят быстро, по другим — медленно.

Идея решения: подбирать индивидуальный шаг обучения для каждого веса. Если параметр уже активно менялся (градиенты по нему были большими), шаг нужно уменьшать; если изменений было мало — шаг остаётся больше

Алгоритм работы метода **AdaGrad (Adaptive Gradient)**:

- Для каждого веса w_j накапливаем сумму квадратов градиентов:

$$G_{j,t} = G_{j,t-1} + g_{j,t}^2, \quad g_{j,t} = (\nabla Q(w_t))_j$$

- Обновляем веса по формуле:

$$w_{j,t} = w_{j,t-1} - \frac{\eta}{\sqrt{G_{j,t} + \varepsilon}} g_{j,t}$$

- чем больше градиенты по координате j накапливались в прошлом ($G_{j,t}$ велико), тем меньше становится текущий шаг — знаменатель увеличивается
- добавка ε нужна для защиты от деления на ноль
- в итоге каждый параметр «подстраивает» свой learning rate, что стабилизирует сходимость и ускоряет обучение.

RMSProp (Root Mean Squared Propagation)

В AdaGrad знаменатель $\frac{\eta}{\sqrt{G_{j,t}} + \epsilon}$ только растёт со временем. Это значит, что шаг обучения постоянно уменьшается. В итоге на длинных обучениях алгоритм может «замереть» задолго до достижения хорошего минимума

Чтобы избежать этой проблемы, в **RMSProp** вместо «вечной суммы» квадратов градиентов берут скользящее среднее:

$$G_{j,t} = \alpha \cdot G_{j,t-1} + (1 - \alpha) \cdot g_{j,t}^2$$

$\alpha \in [0,1]$ — коэффициент сглаживания

Обновление весов:

$$w_{j,t} = w_{j,t-1} - \frac{\eta}{\sqrt{G_{j,t}} + \epsilon} g_{j,t}$$

- Если градиенты по параметру большие, $G_{j,t}$ растёт и шаг уменьшается
- Если градиенты стали меньше, $G_{j,t}$ тоже уменьшается (из-за затухания старых значений) и шаг снова увеличивается

То есть шаг может адаптивно **как уменьшаться, так и увеличиваться**

Adam (Adaptive Moment Estimation)

Adam (Adaptive Moment Estimation) объединяет:

- Импульс (как в Momentum/Nesterov) — учитываем среднее значение градиентов, то есть направление движения
- Адаптивный шаг (как в RMSProp) — учитываем дисперсию градиентов, чтобы нормализовать скорость по каждой координате

Обновление весов:

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{G_{j,t}} + \varepsilon} h_{j,t}$$

Импульс (как в Momentum):

$$h_{j,t} = \frac{\beta_1 \cdot h_{j,t-1} + (1 - \beta_1) \cdot g_{j,t}}{1 - \beta_1^t}$$

Нормализация шага (как в RMSProp):

$$G_{j,t} = \frac{\beta_2 \cdot G_{j,t-1} + (1 - \beta_2) \cdot g_{j,t}^2}{1 - \beta_2^t}$$

β_1 и β_2 — коэффициенты сглаживания

Модификации градиентного спуска

- Метод **Momentum** ускоряет градиентный спуск за счёт «инерции» — накопления прошлых шагов, что помогает не застревать в мелких локальных минимумах и сглаживает колебания
- **AdaGrad** адаптирует скорость обучения под каждый параметр, уменьшая шаг там, где градиенты большие, и позволяя быстрее сходиться, но его недостаток в том, что шаг всегда уменьшается и обучение может остановиться раньше времени
- **RMSProp** решает эту проблему, используя скользящее среднее квадратов градиентов, благодаря чему шаг может как уменьшаться, так и увеличиваться, поддерживая стабильную сходимость
- **Adam** объединяет идеи Momentum и RMSProp: он учитывает и накопленное направление движения, и адаптивный выбор шага, что делает его самым популярным и универсальным методом оптимизации в глубоком обучении.

